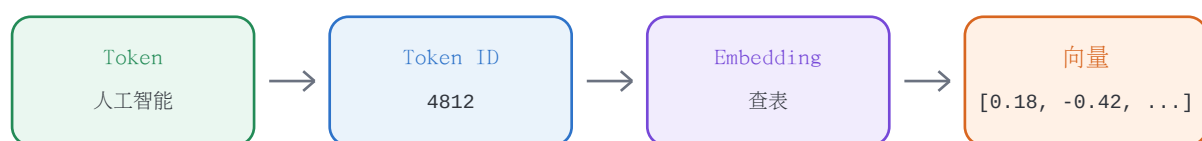


第3讲：Embedding

文字怎样变成“有意义的向量”？

从 Token ID 的“地址”走到神经网络真正能计算的表示



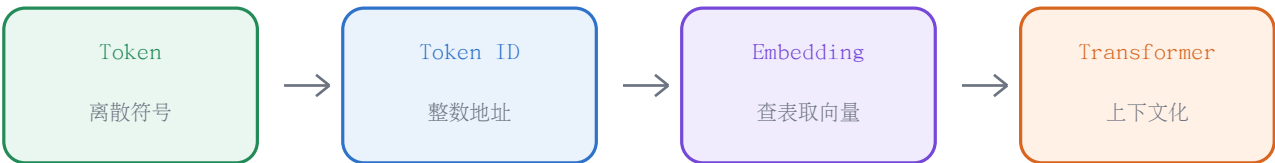
适合对象：零基础。默认不要求线性代数、神经网络、NLP 基础。

本讲原则：先建立“向量=坐标”的直觉，再解释 Embedding 矩阵、相似度、上下文化表示与 RAG。

本讲学习目标

核心目标 学完以后，你应该能完整解释：为什么 Token ID 还不能表示语义；Embedding 是怎样把离散 ID 变成连续向量的；向量相似度为什么能表达“接近”；LLM 内部 Token Embedding 与 RAG 文本 Embedding 有什么区别。

知识地图



你要掌握的层次	本讲要求
直觉层	知道“向量就是一串坐标”，知道为什么要把 ID 映射成向量。
机制层	知道 Embedding Matrix、lookup、维度 d、参数量 $V \times d$ 。
几何层	理解点积、欧氏距离、余弦相似度的直觉。
LLM 层	理解 Token Embedding 只是输入表示；经过 Transformer 后会变成上下文化表示。
Agent/RAG 层	区分内部 Token Embedding 与外部检索 Embedding，理解向量检索的基本原理。

1

先解决一个关键问题：Token ID 为什么还不够？

整数编号能标识 Token，但编号本身没有“语义距离”。

上一讲我们得到了一条链路：文字先被 tokenizer 切成 Token，每个 Token 再对应一个 Token ID。例如为了教学，我们假设“猫”是 104，“狗”是 205，“汽车”是 806。

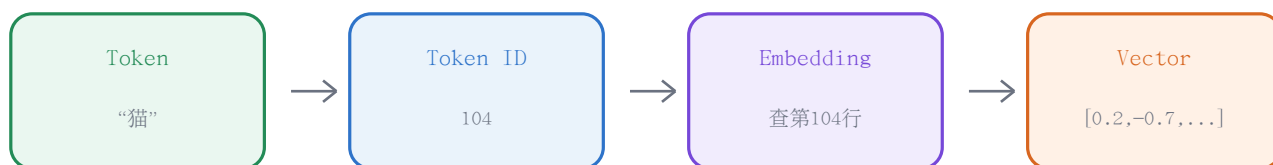
■ -> 104
■ -> 205
■■ -> 806

问题来了：104、205、806 只是编号。205 比 104 大，并不意味着“狗比猫多 101 个单位”；806 离 205 更远，也不意味着“汽车在语义上比狗离猫更远”。

关键比喻 Token ID

更像“图书馆的书号”或“身份证号”：它负责唯一标识对象，而不是表达对象的意义。

所以神经网络不能把 Token ID 直接当成普通数值使用。我们需要一个新的表示层，把“离散编号”映射到“可以参与连续数值计算的向量”。这个表示层就是 Embedding。



科研阅读提醒 论文里常见 token embedding、input embedding、embedding layer、embedding matrix，这几个词在当前上下文里通常围绕同一件事：把离散 token 映射到连续向量。

2

向量到底是什么？

先不要想高等数学：把它看成“一个对象在很多维度上的坐标”。

零基础最容易被“向量”这个词吓住。最简单的理解是：向量就是一串有顺序的数字。

■■■■: [2.0, 3.0]

■■■■: [0.4, -1.2, 5.7]

■■■■: [0.18, -0.42, 0.91, ..., 0.07]

二维时我们还能画在纸上：第一个数字决定横坐标，第二个数字决定纵坐标。高维向量本质没有变，只是“坐标轴”多到人眼无法直接画出来。



二维只是教学示意；真实 Embedding 通常有很多维。

图里“猫”和“狗”靠得比较近，“汽车”和“公交车”靠得比较近。这个图只是教学示意，但它表达了 Embedding 最重要的直觉：模型可以把某些在数据中用法相似的对象，学到向量空间中相近的位置。

不要误解 真实 Embedding 的每一维通常没有“第1维=动物程度、第2维=交通工具程度”这种人类可读标签。语义往往分散在许多维度的组合中。

3

Embedding Layer 的核心：一张可学习的“向量表”

Token ID 像地址，Embedding Matrix 像表格。

假设词表大小 $V=50,000$ ，Embedding 维度 $d=4,096$ 。Embedding 层可以想成一张巨大的矩阵：

$$E \in \mathbb{R}^{(V \times d)}$$

矩阵有 V 行，每一行对应一个 Token；每一行有 d 个数字，这 d 个数字就是该 Token 的起始向量。



Lookup 的直觉：ID 不是拿来“计算语义”的，它更像地址；用它去矩阵里取出一行向量。

当 Token ID=4812 时，并不是把数字 4812 乘进去，而是执行类似：

```
vector = E[4812]  
# ■■ Embedding Matrix ■■ 4812 ■
```

因此，从程序角度看，Embedding 很像“查表 (lookup)”；从学习角度看，这张表里的数字不是人工固定的，而是模型训练过程中一起被调整的参数。

底层原理 Embedding 层之所以能表达语义，不是因为“查表”这个动作神奇，而是因为这张表的数值会在训练中被梯度下降不断修改，使它越来越有利于预测任务。

4

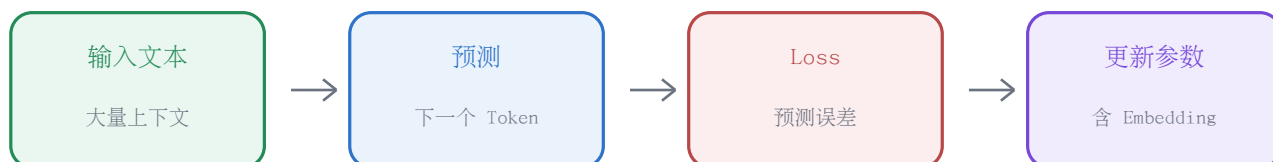
Embedding 是怎么“学出来”的？

不是人手工给猫、狗、汽车填坐标，而是训练目标反向塑造这些坐标。

先看一个极度简化的过程。模型大量看到类似文本：

■ ■ ■ ■
■ ■ ■ ■
■ ■ ■ ■ ■
■ ■ ■ ■ ■

为了完成“预测下一个 Token”等训练任务，模型需要让“猫”“狗”“动物”“吃”等表示组合起来以后有助于正确预测。如果某种向量位置让预测更准确，损失（loss）就会下降；反向传播会推动参数朝这个方向调整。



这意味着“语义相近”并不是训练目标里直接写着的一条规则，而常常是完成预测任务过程中形成的有用结构。

通俗比喻 像城市长期发展：没有人一开始画一张图规定“餐馆必须彼此靠近”，但相似功能、相似客流、相似约束长期作用后，会形成可观察的聚集结构。Embedding 的几何结构也常是训练压力长期塑造出来的。

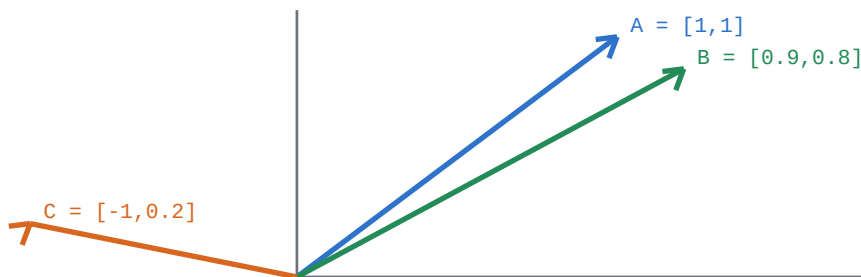
注意：不同模型、不同训练目标、不同 tokenizer、不同数据，得到的 Embedding 空间都可能不同。不能把某一个模型的坐标当成“语言的唯一真实坐标系”。

5

“向量接近”到底怎么算？

距离与相似度是把“几何接近”变成可计算数值的方法。

常见的比较方式有三类：欧氏距离（Euclidean Distance）、点积（Dot Product）、余弦相似度（Cosine Similarity）。对于零基础，最需要掌握的是余弦相似度的直觉。



余弦相似度主要比较“方向是否接近”，而不是向量的绝对长度。

余弦相似度公式：

$$\cos(a, b) = (a \cdot b) / (||a|| \times ||b||)$$

它主要比较两个向量方向是否接近。方向越接近，cosine 越靠近 1；近似垂直时接近 0；方向相反时接近 -1。

小例子：A=[1,1]，B=[0.9,0.8]，两者余弦相似度约为 0.998，说明方向非常接近。

指标	直觉	常见用途
欧氏距离	两点直线距离；越小越近	聚类、几何距离等
点积	方向和长度共同影响	Attention 等大量神经网络计算
余弦相似度	主要看方向是否一致	文本检索、向量检索常见

边界

相似度高不等于“事实相同”“结论正确”“可以互相替代”。它只是某个表示空间中的数值关系。

6

最容易混淆的概念：Token Embedding 不等于最终“语义表示”

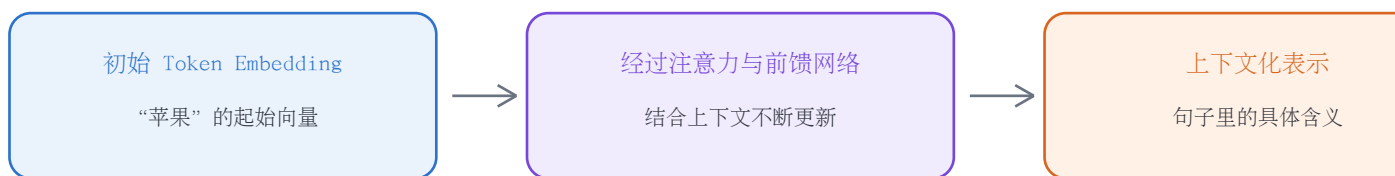
初始向量会在 Transformer 里被上下文不断改写。

考虑两个句子：

■ A: ■■■■■■■■

■ B: ■■■■■■■■

如果 tokenizer 都产生同一个“苹果”Token，那么它进入模型时可以查到同一行初始 Token Embedding。但是进入 Transformer 之后，“苹果”会通过 Self-Attention 与周围 Token 交互。



例：“我吃了一个苹果”与“苹果发布了新设备”中，“苹果”起始 Token Embedding 可以相同，但经过 Transformer 后，其上下文化表示会因为上下文不同而发生明显变化。

因此要区分两个层次：

层次	含义	是否依赖上下文
Token Embedding	输入端查表得到的起始向量	通常不依赖当前句子的具体上下文
Hidden State / Contextual Representation	经过多层 Transformer 后的内部表示	强烈依赖当前上下文

论文阅读能力 以后论文里出现 hidden representation、hidden state、contextual embedding，不要机械地把它都理解成“Embedding 矩阵里那一行”。很多时候指的是 Transformer 中间层或最后层的上下文化表示。

7

Word2Vec 和 LLM 的 Embedding 是一回事吗？

历史上都叫 embedding，但现代 LLM 的表示链更复杂。

早期经典词向量方法（例如 Word2Vec、GloVe）让“一个词对应一个主要向量”成为非常著名的表示方式。这对理解 Embedding 很有帮助，但不要直接把它等同于现代 LLM。

对比	经典静态词向量	现代 LLM
基本单位	通常是 word / token	通常是 subword token
输入起始向量	一个词/Token 对应向量	Token Embedding Matrix 查表
上下文化	弱或没有	经过 Transformer 后强上下文化
同一个词多义	通常共享同一向量	最终 hidden state 会因上下文而变化
主要价值	语义表示、相似度	作为深层序列计算的入口

所以更准确的心智模型是：LLM 的 Token Embedding 是“起点”，不是整个语义理解的终点。

例子 “bank” 可以表示银行，也可以表示河岸。静态词向量往往需要一个向量兼顾这些含义；现代 Transformer 会利用上下文，把不同句子中的 bank 形成不同的上下文化内部表示。

8

Embedding

维度与参数量：为什么一张“表”也很大？

$V \times d$ 会直接贡献模型参数和显存占用。

如果词表大小 $V=50,000$ ，Embedding 维度 $d=4,096$ ，那么 Embedding Matrix 的参数数量就是：

$$50,000 \times 4,096 = 204,800,000$$

也就是约 2.048 亿个可学习数字。若每个参数使用 2 字节保存，仅这张矩阵理论上就约占 409.6 MB（这里是教学估算，真实框架还会有其他开销）。

这说明两个概念：

变量	变大后发生什么
词表 V 变大	矩阵行数增加；可以更细地覆盖符号，但参数量和输出层计算也可能增加。
维度 d 变大	每个 Token 表示容量提高，但每层网络的宽度与计算开销通常也会增大。

有些架构会让输入 Embedding 与输出预测层共享一部分权重，这叫 **weight tying**（权重绑定）。你现在只需要知道：模型设计者会尽量复用参数，降低冗余。

科研提醒 看到论文中的 vocabulary size、hidden size、embedding dimension 时，要意识到它们不只是“配置项”，还会影响参数量、显存、吞吐和系统成本。

9

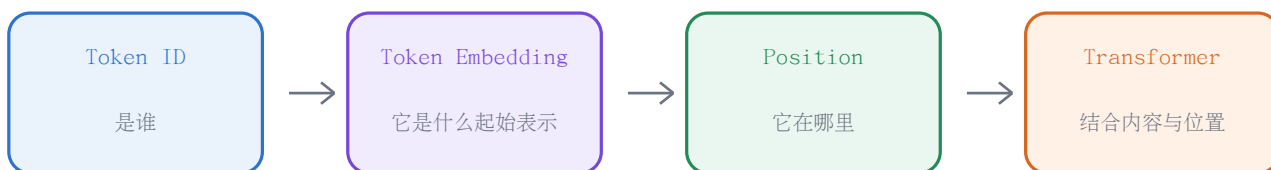
只有 Token Embedding 够吗？还不够：模型还要知道位置

“人咬狗”和“狗咬人”Token 集合相同，但顺序不同。

如果只把每个 Token 独立查表，模型得到的是一组向量，却还没有充分表示它们在序列中的先后位置。



两句话用到的 Token 可能完全相同，但意义不同。因此 Transformer 还需要注入位置信息。早期方法有 Positional Encoding；现代 LLM 中常见 RoPE 等方案。



本讲边界 本讲只建立“Embedding 需要和位置信息一起进入 Transformer”的直觉。位置编码/RoPE 会在后续 Transformer 专门课程里详细讲。

10

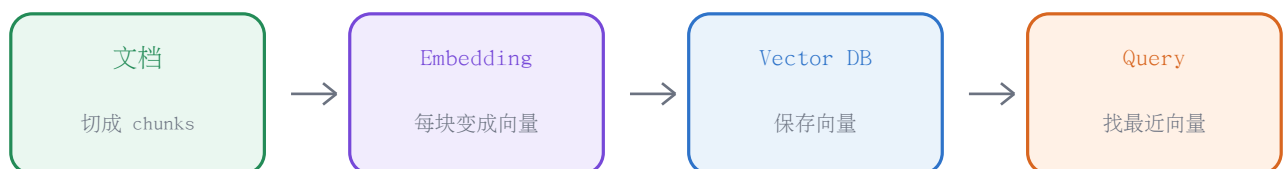
第二种常见 Embedding: RAG / 向量数据库里的文本向量

名字相同，但用途与粒度不同。

以后你做 Agent、RAG、Memory 时，会频繁看到“把一段文本做 embedding，然后放进 vector database”。这时候的 embedding 往往不是“直接拿 LLM 输入层某个 Token 的向量”。

项目	LLM 内部 Token Embedding	检索用 Text Embedding
对象粒度	单个 Token	句子、段落、文档、查询等
主要目的	作为 Transformer 的输入表示	让语义相近文本在向量空间中接近，便于检索
常见输出	每个 Token 一个向量	一段文本一个固定长度向量
典型场景	LLM 前向计算	RAG、语义搜索、Memory Retrieval

RAG 中的基本流程可以概括为：



用户问题也被编码成一个向量，再与数据库里的文档向量比较相似度。相似的文档片段被取回，放入 LLM 上下文，帮助模型回答。

核心区别 Token Embedding 解决“神经网络如何接收离散 Token”；Text Embedding 解决“如何把一段文本压成适合比较和检索的向量”。

11

Embedding 与 Agent Systems 有什么关系？

Memory、工具选择、知识检索、经验检索都可能依赖向量表示。

对普通聊天用户，Embedding 往往是隐藏在底层的细节；但对 Agent Systems 研究者，它经常直接变成系统模块。

Agent 模块	Embedding 可以做什么	潜在问题
Long-term Memory	把旧经历编码成向量，按相似度取回相关记忆	相似≠真正相关；重要但表述不同的记忆可能漏检
Tool Retrieval	工具很多时，根据用户任务检索候选工具	描述质量与 embedding 模型会影响召回
Knowledge/RAG	从外部文档中取回相关片段	chunk 粒度、向量模型、Top-k 都会改变结果
Experience Retrieval	查找过去与当前问题相似的成功/失败案例	可能导致错误经验被反复放大

这也是一个科研入口：Agent 的“记忆”不是人类意义上的神奇记忆，而常常可以被拆成：存什么、怎么编码、怎么检索、检索多少、什么时候遗忘、取回以后如何使用。Embedding 只解决其中“编码与相似性搜索”的一部分。

AI Systems Science 视角 一旦多个 Agent 都依赖相似度检索，Embedding 的偏差会从“单次检索误差”扩散为系统性行为：错误记忆可能被反复召回、多个 Agent 可能检索到同质信息、群体多样性可能因此下降。

12

常见误区：看到漂亮的向量图，最容易产生哪些错觉？

把教学图、可视化投影和真实高维空间区分开。

误区1: Embedding 每一维都有清晰人类含义。

通常不能这样解释。单个概念可能分布在许多维度的组合中。

误区2: 二维散点图就是模型真实的向量空间。通常是 PCA、t-SNE、UMAP

等降维后的投影，会丢失信息，也可能扭曲局部/全局结构。

误区3: 语义相似度高，就代表事实正确。Embedding 衡量的是表示空间相似性，不是真值验证器。

误区4: 两个词向量近，就一定是同义词。

相近可能来自共同语境、主题、功能、句法角色等，不一定是严格同义。

误区5: RAG 的 embedding 与 LLM 的 token embedding 完全一样。

用途与训练目标常不同；不要只因为都叫 embedding 就混为一谈。

误区6: 一个 Token 的意义在进入模型前已经完全确定。起始 embedding

只是输入，真正的上下文文化表示要依赖 Transformer 的层层交互。

13

动手实验：不用任何第三方库，也能理解 Embedding 几何

先用玩具向量建立直觉，再进入真实模型。

材料包中的 `04_embedding_geometry_demo.py` 使用 Python 标准库，包含两个实验。

实验 A: Token ID $\rightarrow$ Embedding Lookup

```
embedding_table = {  
    104: [0.82, 0.76, 0.10], # ■  
    205: [0.79, 0.73, 0.15], # ■  
    806: [0.05, 0.12, 0.91], # ■■  
}
```

```
vector = embedding_table[104]
```

观察点: ID=104 本身没有语义；真正参与比较的是查出来的向量。

实验 B: 计算余弦相似度

```
cos(■, ■) -> ■  
cos(■, ■■) -> ■
```

这不是在模拟真实 LLM

的全部训练过程，而是在隔离一个最小机制：向量空间允许我们把“接近”转成数值。

推荐执行方式 先不要修改代码，直接运行一次；再把“汽车”的向量改得更接近“猫”，观察 cosine similarity 如何变化。你会直观看到：所谓“相似”，最终就是向量几何关系。

14

本讲术语表

以后读论文时，这些词需要做到“看到就知道在说哪一层”。

术语	中文直觉	你应该会解释什么
Embedding	嵌入/向量表示	把离散对象映射成连续向量。
Embedding Matrix	嵌入矩阵	每行对应一个 token 的可学习向量。
Vocabulary Size (V)	词表大小	矩阵有多少行。
Embedding Dimension (d)	嵌入维度	每个向量有多少个数字。
Lookup	查表	根据 token ID 取出矩阵中的一行。
Dot Product	点积	比较方向和长度的基础运算，Attention 中也会用。
Cosine Similarity	余弦相似度	比较两个向量方向是否接近。
Hidden State	隐藏状态	Transformer 层内部的上下文化表示。
Contextual Representation	上下文化表示	同一 token 在不同上下文中可形成不同内部表示。
Text Embedding	文本向量	把句子/段落编码为适合检索比较的向量。
Vector Database	向量数据库	按向量相似度组织与检索数据。

15

练习题：检查你是不是真的理解了

不要只会背“Embedding=向量”。要能解释机制和边界。

1. 为什么 Token ID=100 和 Token ID=101 并不意味着两个 Token 的语义很接近？
2. Embedding Matrix 的“第 4812 行”在直觉上表示什么？
3. 如果 $V=100,000$, $d=2,048$, Embedding Matrix 有多少参数？
4. “苹果公司发布手机”和“我吃了一个苹果”中，“苹果”的 Token Embedding 与最终 hidden state 有什么不同？
5. 余弦相似度高，能不能说明两段文本在事实层面都正确？为什么？
6. LLM 内部 Token Embedding 与 RAG 用的 Text Embedding 最重要的区别是什么？
7. 为什么 Agent Memory 只靠向量相似度检索可能出问题？至少给两个原因。
8. 你看到一张二维“词向量分布图”，为什么不能认为那就是模型真实空间？

建议 先独立回答，再看材料包中的《练习题与答案》。真正掌握的标准不是“认得术语”，而是能用自己的话把 Token ID $\rightarrow$ Embedding $\rightarrow$ Transformer $\rightarrow$ Contextual Representation 讲通。

16

一页总结：把整讲压缩成六句话

如果下面六句能自己解释出来，本讲就过关了。

1. Token ID 只是离散编号，本身没有可直接解释的语义距离。
2. Embedding Layer 用 Token ID 去 Embedding Matrix 中查出一个连续向量。
3. Embedding Matrix 是可学习参数；训练通过预测目标与梯度下降不断调整它。
4. 向量可以用距离、点积、余弦相似度等方式比较，但“相似”不等于“事实正确”。
5. Token Embedding 只是起始表示；经过 Transformer 后会形成依赖上下文的 hidden states。
6. RAG / Agent Memory 中的 Text Embedding 主要服务于语义检索，与 LLM 输入层 Token Embedding 不应混为一谈。

与下一讲的连接

现在你已经知道神经网络真正接收的是一串向量。但这些向量如何被处理、参数又是怎么被学习的？下一讲进入：第4讲《神经网络到底是什么？参数、层、激活函数与“学习”的最小直觉》。

